Security review: tenant isolation in an AI-built Next.js + Supabase SaaS

> **Sample report — fictional company, written to show the method.** "Acme Invoicing" does not exist. The database findings below are real results of this repository's test harness against `policies/broken.sql`; the Next.js findings come from the excerpts in `examples/nextjs-broken/`. Every evidence link points to a file in this repo.

Item	Detail
Client	Acme Invoicing (fictional): B2B invoicing SaaS, one Supabase project, self-serve sign-up
Reviewer	[Reviewer name]
Version reviewed	Database: `policies/broken.sql` · App: `examples/nextjs-broken/`
Review type	Human-led, risk-based. Not a line-by-line review, not a penetration test
Status	Draft for discussion. Sign-off after the re-test in section 7

---

1. Summary

The question the client asked: *can one customer (tenant) see or change another customer's data?*

Yes, in several ways, and the most serious ones need no special skill. Three findings are Critical: the service role key is shipped to the browser, a table of third-party API keys is readable without logging in, and any user can make themselves admin of any organization.

The earlier AI-generated audits were right that RLS is "enabled" and that a direct cross-tenant query on `invoices` is blocked. They missed that the tenant is read from columns and claims the user can edit, and that several paths (a view, an RPC, storage, the service key) skip RLS entirely.

Fixing the Critical and High findings is mostly SQL and configuration, not a rewrite. A fixed policy set is in `policies/fixed.sql`; with it, every cross-tenant test is blocked and every same-tenant action still works (`evidence/results-fixed.json`).

Severity	Findings
Critical	F-01, F-02, F-03
High	F-04, F-05, F-06, F-07, F-08
Medium	F-09, F-10
Low	F-11

2. Scope

In scope

- Postgres schema, RLS policies, grants, views and functions in the exposed schemas (`public`, `storage`)
- How the Next.js app creates Supabase clients and where the service role key is used
- Storage bucket policies for invoice PDFs
- The client's earlier AI-generated audit notes (treated as leads, not as conclusions)

Out of scope (not reviewed)

- UI code, styling, and business rules unrelated to who can access which data
- Billing logic, email templates, third-party integrations beyond where their keys are stored
- Hosting, DNS, CI/CD, dependency vulnerabilities (`npm audit` was not part of this review)
- Load, performance and availability
- Edge Functions (none in this sample; see the checklist for what would be checked)

3. Method

The review spends time where a mistake would cross the tenant boundary, and spends little time where it would not.

1. **Map the tenant boundary.** Which tables hold tenant data, which column says who owns a row, and where the app decides "who is this user and which org are they in". Here: every business table has `org_id`; membership lives in `public.memberships`.
2. **Read the earlier audits as leads.** Each claim is turned into a test (section 5).
3. **Automated sweep** of the database catalog (`checks/lint.sql`) and of the Next.js source (`audit/scan_nextjs.py`). These find the obvious patterns fast. They are heuristics.
4. **Attack tests with two real tenants.** A copy of the schema with two organizations and one user each. As user A, try to read, insert, update and delete tenant B's rows, call the RPCs, read the views and list storage objects, using only what any user has: the public anon key and their own JWT. App-level checks are skipped on purpose. Every attempt runs in a rolled-back transaction; raw results are saved (`evidence/results-broken.json`).
5. **Targeted code reading** of the paths the sweep and tests point to: client creation, route handlers, server actions, anything that touches the service role key.
6. **Rank** each finding (section 4) and write the fix.
7. **Re-test** with the fixes applied. The same tests must show BLOCKED for every attack and ALLOWED for every legitimate same-tenant action.

How findings are ranked

Severity comes from **what crosses the boundary**, then is adjusted for **who can do it**. The app has self-serve sign-up, so "any logged-in user" means anyone with an email address.

Severity	What crosses the tenant boundary
Critical	Credentials, full-database access, or control of another tenant (e.g. becoming its admin)
High	Another tenant's business data read or changed (invoices, customers, files)
Medium	Aggregates or low-sensitivity metadata, or a step that makes a High finding easier to aim
Low	A defect that is not reachable today but becomes reachable after a plausible change
Info	Hardening, no exposure

One level down if the attacker must already be an admin of the target tenant or an insider; one level up if a logged-out visitor can do it with a single request. A static FAIL with no working attack path is ranked by the path, not by the pattern (see F-11).

4. Findings

F-01 — Critical — Service role key is shipped to the browser

What: `NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY` is defined in the env file and used in `lib/supabase.ts`, which a client component imports. Next.js inlines every `NEXT_PUBLIC_` variable into the JavaScript bundle. The service role bypasses RLS, so anyone who opens the browser dev tools gets read and write access to every table of every tenant.

A route handler (`app/api/invoices/route.ts`) also uses this client with an `orgId` taken from the request body and never identifies the caller.

Evidence: `evidence/nextjs-scan-broken.json` (NX1, NX2, NX3). Check S1 in `results-broken.json` shows the service role reading both tenants' invoices: RLS does not apply to it, by design.

Fix: rotate the key now (it must be treated as leaked). Rename to `SUPABASE_SERVICE_ROLE_KEY` (no `NEXT_PUBLIC_`), create the admin client in a file that starts with `import 'server-only'`, and use the user's session client (anon key + cookies) everywhere else. Every server path that still needs the service role must call `auth.getUser()` and check membership first. Pattern: `examples/nextjs-fixed/`.

F-02 — Critical — Third-party API keys readable without logging in

What: `public.integration_secrets` was added by a later migration and RLS was never enabled. Supabase's default grants give `anon` and `authenticated` full access to new tables in `public`, so the public anon key reads every tenant's secrets.

Evidence: R5: logged-out request returned both organizations' secrets. Lint: `table_without_rls public.integration_secrets`.

Fix: enable RLS, add no policies, `revoke all ... from anon, authenticated`. Read it only from server code. Rotate every stored secret. Consider Supabase Vault or the hosting platform's secret store instead of a table.

F-03 — Critical — Any user can make themselves admin of any organization

What: the `memberships` INSERT policy only checks `user_id = auth.uid()`. Nothing limits `org_id` or `role`. One request adds the caller as `admin` of tenant B, and every policy that reads `memberships` (today: organizations and project inserts; after the fix: all of them) then treats the caller as B's admin.

Evidence: W5: the row `(org B, alice, admin)` exists after the insert. Lint flagged the policy only as REVIEW (no tenant reference); the attack test confirmed it.

Fix: no INSERT/UPDATE/DELETE policy on `memberships` for `authenticated`. Invites go through a server route that checks the caller is an admin of that org, then writes with the service role (`examples/nextjs-fixed/app/api/invites/route.ts`).

F-04 — High — Tenant is read from a column the user can edit

What: the `invoices` policies use `org_id = (select org_id from profiles where id = auth.uid())`. The `profiles` UPDATE policy lets users update every column of their own row, including `org_id`. Set it to tenant B and all of B's invoices become readable and editable.

Evidence: R2 (direct read of B's invoices) is **blocked**, which is what the earlier audit tested. R3 (update own profile, then read) **leaks**. No static check flags this policy: it mentions `org_id`, it just trusts the wrong source.

Fix: derive the tenant from `memberships` through a helper in a non-exposed schema (`private.is_member(org_id)`), and restrict profile updates to safe columns with a column grant (`grant update (full_name) on profiles to authenticated`).

F-05 — High — RPC returns any organization's invoices, even to logged-out visitors

What: `public.get_org_invoices(p_org_id)` is `SECURITY DEFINER`, so it runs as the owner and skips RLS. It trusts the org id argument, has no pinned `search_path`, and `anon` can execute it (Supabase's default function grants).

Evidence: R7 (as user A) and R7b (logged out) both return tenant B's invoices. Lint: `security_definer_in_exposed_schema`, `function_executable_by_anon`.

Fix: `SECURITY INVOKER` so RLS filters the result, `set search_path = ''`, revoke EXECUTE from `public` and `anon`. If a function must be `SECURITY DEFINER`, put it in a schema the API does not expose and have it answer only about `auth.uid()`, never about an id from the caller.

F-06 — High — Invoice PDFs: private bucket, but no tenant check

What: the bucket is private, but the storage policies only check `bucket_id`. Any logged-in user can list and download every tenant's invoice PDFs and upload files into another tenant's folder.

Evidence: R8 (list B's files) and W6 (upload into B's folder) both succeed.

Fix: require the first folder of the path to be an org the caller belongs to: `(storage.foldername(name))[1] in (select id::text from private.my_org_ids() as t(id))`, for SELECT and INSERT (and UPDATE/DELETE if used).

F-07 — High — Cross-tenant writes on invoices and projects

What: three write policies check "logged in" or "is an admin somewhere" instead of "belongs to this row's org":

- `invoices` INSERT: `with check (auth.uid() is not null)`: W1 inserts an invoice into B.
- `projects` UPDATE: `using (auth.role() = 'authenticated')`: W3 renames B's project.
- `projects` DELETE: admin of *any* org: W4 deletes B's project.

Evidence: W1, W3, W4. Lint flags the first two as FAIL and the third as REVIEW.

Fix: every write policy pins the row to a tenant the caller belongs to, in both USING and WITH CHECK (`private.is_member(org_id)`, `private.is_admin(org_id)` for deletes).

F-08 — High — Customer list scoped by `user_metadata`

What: the `customers` SELECT policy reads the tenant from `auth.jwt() -> 'user_metadata'`. Supabase users can change their own `user_metadata` with `auth.updateUser({ data: ... })`, and the new value is in their next token. Customer names and emails of any tenant are exposed.

Evidence: R4 (token with self-edited `user_metadata.org_id`) reads B's customers. Lint: `policy_trusts_user_metadata`. The harness simulates the edited token; the `updateUser` call itself was not run here (no Supabase Auth server in the lab).

Fix: use `memberships`. If a claim is really needed, use `app_metadata` (writable only with the service role) or a custom access token hook.

F-09 — Medium — `invoice_summary` view skips RLS

What: a plain view runs with its owner's rights. Any logged-in user reads the invoice count and total of every tenant. Aggregates only, no row data, but it reveals each customer's revenue through the product.

Evidence: R6. Lint: `view_without_security_invoker`.

Fix: `create view ... with (security_invoker = true)` (Postgres 15+), revoke from `anon`.

F-10 — Medium — Projects readable by everyone, including logged-out visitors

What: the dashboard-template policy `"Enable read access for all users" ... using (true)` with no `TO` clause. Project names are low-sensitivity, but the rows include `org_id`, which is exactly the input F-05 needs.

Evidence: R1 (user A reads B's projects), R1b (logged-out visitor reads all projects). Lint: `policy_always_true`.

Fix: `to authenticated using (private.is_member(org_id))`.

F-11 — Low — Invoice UPDATE policy has `WITH CHECK (true)`

What: the policy lets a row's new values be anything. A filtered update that moves an invoice to tenant B is blocked today (W2), only because Postgres also checks the new row against the SELECT policy when an UPDATE reads columns. An unfiltered update running as the user is not (W2b moves all of the caller's invoices into B).

Why Low: no path in the reviewed app sends an unfiltered update, so it is not reachable today; it becomes reachable as soon as the SELECT policy or a SQL function changes. Lint reports it as FAIL; the rank follows the attack path, not the pattern.

Fix: `with check (private.is_member(org_id))`.

Closed during review (no finding)

- `profiles` SELECT/UPDATE policies flagged REVIEW by lint (no tenant reference): they are per-user, which is correct for that table. Closed after reading.
- Same-tenant actions (read, create, update, RPC, storage) checked as working in both versions (L1–L7), so the fixes do not break the product.

5. Earlier AI audit claims, checked

Claim in the earlier AI-generated audit (fictional)	Verified?	Evidence
"RLS is enabled on all tables"	**No.** `integration_secrets` has no RLS	R5, lint
"All policies are scoped by `org_id`"	**Misleading.** They mention `org_id`, but read it from `profiles.org_id` and `user_metadata`, which the user controls	R3, R4
"Users cannot read other organizations' invoices"	**True for a direct query only**	R2 blocked, R3/R6/R7 leak
"The service role key is only used server-side"	**No.** It has a `NEXT_PUBLIC_` name and a client component imports it	NX1, NX2
"Storage bucket is private"	**True, and not enough.** Private means "needs a policy", and the policy has no tenant check	R8, W6

6. Recommended order of work

1. **Today:** rotate the service role key and every secret in `integration_secrets` (F-01, F-02). Remove the `NEXT_PUBLIC_` service key and redeploy.
2. **Next:** apply the membership-based policies (F-03, F-04, F-07, F-08, F-10), the RPC and view changes (F-05, F-09) and the storage policies (F-06). In this sample they are one file: `policies/fixed.sql`.
3. **Then:** F-11, and add the attack tests to CI so a future AI-generated migration that reopens a hole fails the build.
4. **Check logs** (Supabase API and storage logs) for past use of the leaked paths, for the period the key and the open table were exposed. This review did not do that.

7. Re-test and sign-off

Re-test against `policies/fixed.sql` and `examples/nextjs-fixed/`:

- every attack in the matrix: **blocked** (`evidence/results-fixed.json`)
- every same-tenant action: **allowed**
- lint: no FAIL; the only REVIEW items are the per-user `profiles` policies, closed above (`evidence/lint-fixed.json`)
- Next.js scan: no findings (`evidence/nextjs-scan-fixed.json`)

Sign-off statement (template)

> I reviewed the tenant isolation of [product] at [commit / migration id], within the scope and method described in sections 2 and 3. As of [date], every Critical and High finding in this report has been fixed and re-tested with the attack tests listed in section 7; the results are attached. Findings [ids] were accepted by [name, role] for the reasons recorded in [link].
>
> This is a risk-based review, not a line-by-line audit or a penetration test. It does not cover the items listed as out of scope, and it reflects the code at the version named above only. Later changes,

> including AI-generated migrations, need the same tests re-run.
>
> [Reviewer name] · [date]

8. Limits of this review

- Tests ran against plain Postgres 16 configured to behave like Supabase (roles, `auth.uid()`, default grants, storage tables), not against a hosted Supabase project. The HTTP layer (PostgREST), Supabase Auth and the Storage API were not in the loop.
- Public buckets (served by URL without policies), signed URL expiry, Realtime and GraphQL were not tested.
- The Next.js scan is a heuristic over source files. It did not inspect a production build (`.next/static`) or deployed environment variables.
- No review of dependencies, infrastructure, rate limiting, or abuse of legitimate features.
- A clean result means these tests found nothing, not that nothing exists.